



Research Intake 2026

Day 19

BERT, RoBERTa, ALBERT & Applications

A Beginner's Guide for Students

From foundational concepts to research-level understanding

Gudsky Research Foundation

Table of Contents

1	Pre-training Paradigm: Why It Works	4
1.1	The Two-Stage Transfer Learning Pipeline.....	4
1.2	Self-Supervised Learning: Labels from the Data Itself.....	4
1.3	Why Bidirectional Context Matters	5
1.4	The Role of Scale.....	5
2	BERT: Bidirectional Encoder Representations from Transformers	6
2.1	Masked Language Modelling (MLM)	6
	The Masking Procedure	6
	Why Not Mask All 15%?.....	7
2.2	Next Sentence Prediction (NSP)	7
2.3	Input Representation: CLS, SEP, and Token Types	8
	Token Embeddings	8
	Segment Embeddings.....	8
	Positional Embeddings.....	8
	Special Tokens: [CLS] and [SEP].....	8
3	BERT Fine-Tuning on Downstream Tasks.....	9
3.1	Single-Sentence Classification.....	9
3.2	Sentence-Pair Classification (NLI, Paraphrase Detection)	9
3.3	Token-Level Tasks: Named Entity Recognition.....	10
3.4	Extractive Question Answering	10
3.5	Practical Fine-Tuning Guidelines	11
4	RoBERTa: Robustly Optimised BERT Pretraining Approach	12
4.1	Dynamic Masking and the Removal of NSP	12
	Static vs. Dynamic Masking	12
	Removing NSP.....	12
4.2	Larger Batches, More Data, and Longer Training	12
	Training Data Scale.....	12
	Larger Batch Sizes	13
	Longer Sequences	13
5	ALBERT: A Lite BERT for Self-Supervised Learning of Language Representations	14

5.1	Factorised Embedding Parametrisation	14
5.2	Cross-Layer Parameter Sharing	14
	Does Sharing Hurt Performance?	15
	Sentence Order Prediction (SOP)	15
6	DistilBERT and Knowledge Distillation	16
6.1	Knowledge Distillation: The Core Idea	16
6.2	DistilBERT Architecture and Training	16
6.3	Performance and Deployment Trade-offs	17
7	BERT-Based Applications In Depth	18
7.1	Search and Information Retrieval	18
7.2	Biomedical and Clinical NLP	18
7.3	Legal and Financial NLP	19
7.4	Multilingual and Cross-Lingual BERT	19
7.5	Conversational AI and Question Answering Systems	19
8	GLUE and SuperGLUE Benchmarks	21
8.1	GLUE: General Language Understanding Evaluation	21
8.2	SuperGLUE: A Harder Benchmark	21
8.3	Key BERT-Family Results on GLUE and SuperGLUE	22
9	Research Frontiers: Probing BERT — What Does It Know?	23
9.1	Probing Classifiers: The Methodology	23
9.2	What BERT Knows: Empirical Findings	24
	Syntactic Knowledge	24
	Semantic Knowledge	24
	World Knowledge	24
9.3	What BERT Does Not Know: Limitations	24
9.4	Attention Analysis: What Does BERT Attend To?	25
9.5	Implications for Research and Practice	25
9.6	Summary: BERT, RoBERTa, ALBERT, and DistilBERT at a Glance	26

1 Pre-training Paradigm: Why It Works

The pre-training paradigm represents one of the most transformative shifts in the history of natural language processing. Before its emergence, NLP models were trained almost entirely from scratch on task-specific labelled datasets — a process that demanded enormous quantities of annotated data, produced brittle models that generalised poorly, and required separate architectures for every new task. The paradigm shift began in earnest with ELMo (Peters et al., 2018) and reached full maturity with the publication of BERT (Devlin et al., Google AI Language, 2018).

The core intuition is elegant: human language is not random. Across millions of documents, words follow predictable statistical patterns, sentences obey grammatical constraints, and meaning is encoded in the relationships between tokens. A model trained on a sufficiently large and diverse corpus of raw text — even without any explicit labels — will internalise an extraordinary amount of linguistic knowledge: syntax, semantics, co-reference, world facts, and even pragmatic reasoning. This learned representation can then be transferred to a specific downstream task with minimal additional supervision.

1.1 The Two-Stage Transfer Learning Pipeline

The pre-training paradigm in NLP follows a two-stage pipeline that has become standard practice across virtually all modern language models:

- **Stage 1 — Pre-training:** A large transformer model is trained on a massive unlabelled text corpus (Wikipedia, BooksCorpus, CommonCrawl, etc.) using self-supervised objectives — tasks whose labels are derived automatically from the data itself. The model learns to predict masked tokens, next sentences, or other properties purely from the raw text, with no human annotation required.
- **Stage 2 — Fine-tuning:** The pre-trained model, with all its learned representations intact, is adapted to a specific downstream task (sentiment analysis, named entity recognition, question answering, etc.) by training on a small labelled dataset. Because the model already understands language deeply, it requires far fewer labelled examples to achieve strong performance than a model trained from scratch.

This two-stage approach solves three longstanding problems in NLP simultaneously: the data bottleneck (labelled data is expensive; unlabelled text is abundant), the generalisation problem (representations learned from broad pre-training transfer better), and the computational efficiency problem (train once, fine-tune many times).

1.2 Self-Supervised Learning: Labels from the Data Itself

The pre-training objectives used in BERT and its variants are examples of self-supervised learning — a class of methods that generate supervisory signals automatically from the input data. This is critical because it removes the need for human annotation during the expensive pre-training phase.

Consider the Masked Language Model (MLM) objective: if we take the sentence 'The cat sat on the [MASK]', the correct answer 'mat' can be derived automatically from the original text. No human had to label anything — the label was already there before the mask was applied. Billions of such training examples can be generated from any large text corpus, giving the model an effectively unlimited source of learning signal.

This is philosophically distinct from unsupervised learning (which typically finds clusters or structure without any labels) and supervised learning (which requires human-provided labels). Self-supervised learning occupies a productive middle ground: it uses labels, but those labels are free.

1.3 Why Bidirectional Context Matters

A central design decision in BERT — and a key reason for its superiority over earlier language models — is the use of bidirectional context during pre-training. Earlier autoregressive models like GPT processed text strictly left-to-right: when predicting the next word, the model could only attend to previous tokens. This is appropriate for generation tasks but sub-optimal for comprehension tasks, where the full sentence context is available.

Consider the sentence: 'The bank by the river was flooded.' The word 'bank' is ambiguous — it could refer to a financial institution or a river bank. A left-to-right model processing 'The bank by the...' has not yet seen the disambiguating word 'river'. A bidirectional model, processing the entire sentence simultaneously, can attend to 'river' on the right while encoding 'bank' in the middle, producing a context-sensitive representation that correctly captures the river-bank meaning.

BERT achieves this through the Masked Language Model objective and the Transformer encoder architecture, which uses self-attention — an operation that allows every token to attend to every other token in the sequence simultaneously, regardless of distance or direction.

1.4 The Role of Scale

Scale is not merely a quantitative difference in the pre-training paradigm — it produces qualitatively different capabilities. BERT-Base, with 110 million parameters and trained on 3.3 billion words, demonstrated strong performance on a wide range of NLP benchmarks. BERT-Large, with 340 million parameters, performed substantially better on tasks requiring deeper reasoning. RoBERTa, trained on 160 GB of text with longer sequences, outperformed BERT-Large despite using the same architecture. The consistent empirical finding is that more data, more compute, and more parameters yield better representations, up to the limits of the pre-training objective and architecture.

Key Insight: The Pre-training Revolution

Before pre-training: training a state-of-the-art NER model required 100,000+ labelled sentences.

After pre-training: fine-tuning BERT on 1,000 labelled sentences matches or exceeds the old state of the art.

This 100× reduction in labelled data requirements democratised NLP research — universities, startups, and researchers in low-resource settings could now build competitive NLP systems without massive annotation budgets.

2 BERT: Bidirectional Encoder Representations from Transformers

BERT (Devlin, Chang, Lee & Toutanova, Google AI Language, 2018) is a pre-trained deep bidirectional transformer encoder that achieved state-of-the-art results on eleven NLP tasks upon its release, including the GLUE benchmark, SQuAD question answering, and multiple other datasets. Its publication triggered an upheaval in NLP research comparable in scale to the introduction of deep learning in computer vision, with hundreds of follow-up papers, domain-specific variants, and application systems published within the first two years alone.

BERT is architecturally a standard Transformer encoder stack — the same encoder used in the original 'Attention Is All You Need' paper (Vaswani et al., 2017). What distinguishes BERT is not its architecture per se, but its pre-training objectives and the scale at which it was trained. The model comes in two standard configurations:

Property	BERT-Base	BERT-Large
Transformer Layers (L)	12	24
Hidden Size (H)	768	1024
Attention Heads (A)	12	16
Total Parameters	110 Million	340 Million
Pre-training Data	BooksCorpus + Wikipedia (3.3B words)	BooksCorpus + Wikipedia (3.3B words)
Pre-training Time	~4 days on 16 TPUs	~4 days on 64 TPUs

2.1 Masked Language Modelling (MLM)

The Masked Language Model (MLM) is BERT's primary pre-training objective and its most significant innovation. Standard language model pre-training — as used in GPT and ELMo — predicts tokens sequentially, either left-to-right or as a bidirectional combination of two separate unidirectional models. MLM enables genuine deep bidirectionality in a single model pass.

The Masking Procedure

During pre-training, 15% of the WordPiece tokens in each input sequence are selected at random for prediction. Of these selected tokens, the following replacement strategy is applied:

- **80% — [MASK] replacement:** The token is replaced with the special [MASK] token. This is the standard case: the model must predict the original token from bidirectional context.
- **10% — Random token replacement:** The token is replaced with a randomly chosen vocabulary token. This prevents the model from learning that [MASK] always indicates a token to be predicted, which would create a mismatch at fine-tuning time (since [MASK] never appears in real input sequences).
- **10% — Unchanged:** The token is left as-is. The model still needs to output the correct token for this position, forcing it to maintain useful representations even for unmasked tokens.

Only the masked positions contribute to the pre-training loss — the model is not penalised for its outputs at the 85% of tokens that were not selected. This is computationally efficient but means that each training example produces a relatively weak learning signal compared to predicting every token.

```
# Example: MLM masking procedure
Original: The capital of France is Paris
Token IDs: 1823 4852 119 1441 117 3000

# 15% of tokens selected → 'capital' (idx 1) and 'Paris' (idx 5)
# 80% → [MASK]: 'capital' → [MASK]
# 10% → random: 'Paris' → 'Tokyo' (random token injected)
# 10% → unchanged: (none selected this time)

Input: The [MASK] of France is Tokyo
Target: - capital - - - Paris
Loss: only computed at positions 1 and 5
```

Why Not Mask All 15%?

The 80/10/10 split is not arbitrary — it is a carefully calibrated engineering choice. If 100% of selected tokens were replaced with [MASK], the model would learn a shortcut: only produce meaningful representations for [MASK] tokens, since only those positions affect the loss. This would cause the model to produce poor representations for regular tokens at fine-tuning time, where [MASK] is never present.

The 10% random replacement forces the model to maintain a distributional representation for every token, because any token might be 'wrong' and need correcting. The 10% unchanged forces the model to be robust even when the input appears correct. Together, these components ensure that the model cannot ignore any token position.

2.2 Next Sentence Prediction (NSP)

BERT is trained with a second objective, Next Sentence Prediction (NSP), designed to help the model understand relationships between sentences — a capability required for tasks like natural language inference and question answering, where two sentences must be jointly analysed.

During pre-training, 50% of training pairs consist of two adjacent sentences from the corpus (positive examples, labelled IsNext), and 50% consist of a sentence from the corpus paired with a randomly chosen sentence from a different document (negative examples, labelled NotNext). The model must predict, from the [CLS] token representation, which class the pair belongs to.

⚠ NSP — A Controversial Design Choice

Later work demonstrated that NSP may be counterproductive. Because negative pairs are drawn from different documents, they are trivially distinguishable by topic — not by discourse structure.

RoBERTa (Liu et al., 2019) ablated NSP and found consistent performance improvements across benchmarks.

ALBERT (Lan et al., 2019) replaced NSP with Sentence Order Prediction (SOP), a harder objective.

The consensus in 2024 is that NSP as formulated in BERT is a weak objective that adds noise.

2.3 Input Representation: CLS, SEP, and Token Types

BERT's input representation is a critical design element that enables the model to handle both single-sentence and sentence-pair tasks within a unified architecture. Every input to BERT is constructed from three additive embedding components:

Token Embeddings

Each token in the vocabulary is mapped to a learned embedding vector of dimension H (768 for BERT-Base). BERT uses WordPiece tokenisation, a subword segmentation algorithm that splits rare or unknown words into smaller known sub-units. For example, 'unaffable' might be tokenised as ['un', '##aff', '##able']. The '##' prefix indicates a continuation piece. This allows BERT to handle an open vocabulary without an excessively large embedding matrix — the standard BERT vocabulary contains approximately 30,000 WordPiece tokens.

Segment Embeddings

BERT processes sentence pairs for tasks like NSP, textual entailment, and question answering. To distinguish the two sentences in the pair, BERT adds a learned segment embedding to every token: Embedding A for all tokens in the first sentence and Embedding B for all tokens in the second sentence. There are only two segment embeddings in standard BERT. For single-sentence tasks, all tokens receive Embedding A.

Positional Embeddings

The Transformer self-attention mechanism is permutation-invariant — it treats the input as a set of tokens, not a sequence. To inject order information, BERT adds learned positional embeddings to each token. BERT uses absolute positional embeddings, with positions 0 through 511 (BERT supports sequences up to 512 tokens). The positional embeddings are learned during pre-training, not fixed sinusoidal encodings as in the original Transformer paper.

Special Tokens: [CLS] and [SEP]

[CLS] (Classification token): Every input sequence begins with the special [CLS] token. After processing through all transformer layers, the hidden state corresponding to [CLS] serves as the aggregate sequence representation for classification tasks. During pre-training, [CLS] is used for NSP. During fine-tuning on classification tasks, a linear layer is added on top of [CLS]'s final hidden state. [SEP] (Separator token): The [SEP] token is used to separate sentences in sentence-pair inputs. For a pair (Sentence A, Sentence B), the input format is: [CLS] Sentence A [SEP] Sentence B [SEP]. The trailing [SEP] after Sentence B marks the end of the sequence.

```
# Single sentence input (e.g., sentiment classification)
[CLS] The movie was fantastic [SEP]
A  A  A  A  A  A  ← Segment embeddings
0  1  2  3  4  5  ← Positional embeddings

# Sentence pair input (e.g., NLI, QA, NSP)
[CLS] Sentence A tokens [SEP] Sentence B tokens [SEP]
A  A  A  A  A  B  B  B  B  ← Segments
0  1  2  3  4  5  6  7  8  9  10 ← Positions

# Final input embedding = Token Embedding + Segment Embedding + Position Embedding
```


3 BERT Fine-Tuning on Downstream Tasks

Fine-tuning BERT on downstream tasks is architecturally straightforward but conceptually profound. The pre-trained model serves as a powerful universal language encoder; fine-tuning adapts this encoder to a specific task by adding a small task-specific output layer and training the entire system end-to-end on labelled data. The key insight is that the pre-trained weights already encode deep linguistic knowledge — fine-tuning merely steers that knowledge toward the target task.

Across all fine-tuning scenarios, the procedure is consistent: take the pre-trained BERT model, add a task-specific head (a linear classifier, a span predictor, etc.), and train on the downstream dataset with a small learning rate (typically $2e-5$ to $5e-5$) to avoid catastrophically forgetting the pre-trained representations. Fine-tuning typically requires only 2–4 epochs on task-specific data.

3.1 Single-Sentence Classification

For tasks that require assigning a label to an entire sentence — sentiment analysis (positive/negative), grammatical acceptability (CoLA), topic classification — BERT is fine-tuned by adding a linear classification layer on top of the [CLS] token's final hidden state.

If the [CLS] hidden state has dimension H and there are K output classes, the task head is a single linear layer $W \in \mathbb{R}^{(K \times H)}$. The probability of each class is computed via softmax, and the model is trained to minimise cross-entropy loss. The softmax over K classes uses the standard formulation: $P(\text{class } k \mid \text{input}) = \exp(W_k \cdot h_{\text{CLS}}) / \sum_j \exp(W_j \cdot h_{\text{CLS}})$.

```
# Fine-tuning BERT for sentiment classification (PyTorch)
from transformers import BertForSequenceClassification, BertTokenizer
import torch

tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = BertForSequenceClassification.from_pretrained(
    'bert-base-uncased', num_labels=2 # positive / negative
)

inputs = tokenizer('The film was absolutely brilliant!',
                  return_tensors='pt', padding=True, truncation=True)
outputs = model(**inputs, labels=torch.tensor([1])) # 1 = positive
loss = outputs.loss
logits = outputs.logits # shape: [batch_size, num_labels]
```

3.2 Sentence-Pair Classification (NLI, Paraphrase Detection)

Tasks that require comparing two sentences — Natural Language Inference (determine if Premise entails, contradicts, or is neutral with respect to Hypothesis), semantic textual similarity, paraphrase detection — are handled by feeding both sentences as a pair in the [CLS] Sentence A [SEP] Sentence B [SEP] format. The [CLS] representation then encodes the relationship between both sentences, and the same linear classification head is applied.

BERT's NSP pre-training objective was specifically intended to develop this inter-sentence understanding capability, though as discussed in Section 4, subsequent research found NSP to be a weak objective for this purpose. Despite this, BERT achieves strong performance on NLI tasks, suggesting that the MLM objective

itself, combined with attending to both sentences simultaneously, is sufficient to capture inter-sentence relationships.

3.3 Token-Level Tasks: Named Entity Recognition

For tasks requiring a label for each token — Named Entity Recognition (NER), Part-of-Speech tagging, chunking — BERT is fine-tuned by applying an independent linear classification layer to the final hidden state of each token (not just [CLS]). If there are K entity/tag classes, the output for each token position i is: $P(\text{tag}_k | \text{token}_i) = \text{softmax}(W \cdot h_i)$.

One important consideration is the mismatch between WordPiece tokenisation and the word-level labels in NER datasets. When a word is split into multiple sub-tokens (e.g., 'Washington' → ['Washington', '##ton']), the label is typically assigned to the first sub-token, and the subsequent sub-tokens are ignored in the loss computation. This avoids double-counting but means the model must produce a correct label at the first sub-token position.

Fine-Tuning Task	Input Format	Output Head	Loss
Sentiment / Topic Classification	[CLS] sentence [SEP]	Linear on [CLS]	Cross-entropy
NLI / Paraphrase	[CLS] sent_A [SEP] sent_B [SEP]	Linear on [CLS]	Cross-entropy
Named Entity Recognition	[CLS] tokens [SEP]	Linear on each token	Cross-entropy
Extractive QA (SQuAD)	[CLS] question [SEP] passage [SEP]	Start/End span logits	Cross-entropy
Semantic Similarity (STS-B)	[CLS] sent_A [SEP] sent_B [SEP]	Linear regression on [CLS]	MSE

3.4 Extractive Question Answering

Extractive QA — where the answer to a question is a contiguous span within a given passage — is one of BERT's most impactful fine-tuning applications, demonstrated on the SQuAD 1.1 and 2.0 benchmarks. The question and passage are packed into the sentence-pair format: [CLS] question [SEP] passage [SEP].

Two learned vectors S (start) and E (end), each of dimension H , are introduced. For each token position i in the passage, the probability of being the start of the answer span is computed as $\text{softmax}(S \cdot h_i)$ over all passage tokens. Similarly for end positions. The predicted answer span is the $[i, j]$ pair that maximises $S \cdot h_i + E \cdot h_j$ subject to $j \geq i$. This is a powerful but simple mechanism — no separate retrieval or pointer network is needed.

Real-World Impact: BERT on SQuAD 2.0

SQuAD 2.0 is a challenging QA benchmark that includes unanswerable questions — the model must recognise

when the passage does not contain a sufficient answer, rather than always producing a span.

Pre-BERT human performance on SQuAD 2.0: 86.8 F1. BERT-Large achieves 83.1 F1 on first release.

With minor modifications (DocQA-style layers, data augmentation), subsequent systems surpassed human performance.

This was the first time a machine had matched human-level performance on an open-domain reading comprehension task.

3.5 Practical Fine-Tuning Guidelines

Based on the original BERT paper and extensive subsequent empirical analysis, the following hyperparameter ranges are recommended for fine-tuning:

- **Learning rate:** 2e-5, 3e-5, or 5e-5 with a linear warmup and decay schedule. Higher learning rates cause catastrophic forgetting; lower rates converge too slowly.
- **Batch size:** 16 or 32. Larger batches improve gradient estimate quality but require more GPU memory.
- **Epochs:** 2, 3, or 4. Fine-tuning for too many epochs leads to overfitting on small datasets; BERT learns quickly from pre-trained representations.
- **Max sequence length:** 128 for most tasks; 512 for long-document tasks (at much higher memory cost).
- **Warmup steps:** 10% of total training steps is a common heuristic for the linear warmup phase.

4 RoBERTa: Robustly Optimised BERT Pretraining Approach

RoBERTa (Liu et al., Facebook AI & University of Washington, 2019) is not a new architecture — it is the same Transformer encoder stack as BERT. Its contribution is empirical: a systematic replication and ablation study of BERT's pre-training procedure, demonstrating that BERT was substantially undertrained and that several design choices could be improved. The title captures the finding precisely: BERT's performance could be robustly optimised by changing how it is pre-trained, not what it is pre-training.

The RoBERTa paper is an important methodological example for researchers: it demonstrates that careful ablation studies on training procedures, not just architectural innovations, can yield substantial performance improvements. The finding that BERT was undertrained — that more data, larger batches, and longer training consistently improve results — had major implications for how subsequent language models were designed and trained.

4.1 Dynamic Masking and the Removal of NSP

Static vs. Dynamic Masking

BERT uses static masking: the masking pattern for each training example is determined once during data pre-processing and then fixed for all training epochs. This means the model sees the same masked version of each sentence in every epoch, potentially memorising the masking pattern rather than learning to generalise.

RoBERTa adopts dynamic masking: the masking pattern is generated on-the-fly every time a training example is fed to the model. Because RoBERTa trains for many more steps than BERT (500K steps vs. 1M steps, on much more data), with dynamic masking each training sequence receives a different mask in every epoch, effectively multiplying the diversity of the training signal. Liu et al. (2019) show that dynamic masking matches or slightly outperforms static masking.

Removing NSP

RoBERTa ablates the NSP objective entirely, finding that removing it consistently improves downstream task performance. The paper proposes three alternative input format strategies and compares them against BERT's segment-pair NSP training:

- **Segment-Pair + NSP (BERT original):** Two segments, possibly spanning multiple sentences, with NSP loss. Each segment is up to 512 tokens.
- **Sentence-Pair + NSP:** Two individual sentences with NSP. Poor performance — too little context per example.
- **Full-Sentences (no NSP):** Inputs are packed from one or more documents, up to 512 tokens, crossing document boundaries with a separator. No NSP loss. Best or near-best performance.
- **Doc-Sentences (no NSP):** Same as full-sentences but sequences do not cross document boundaries. Slightly better on some tasks but requires variable-batch-size training.

The conclusion is clear: the NSP objective adds noise and the sentence-pair constraint limits the amount of context per example. Removing NSP and packing full sequences from documents improves performance across all GLUE and SuperGLUE tasks tested.

4.2 Larger Batches, More Data, and Longer Training

Training Data Scale

BERT was trained on 16 GB of text (Wikipedia + BooksCorpus). RoBERTa was trained on 160 GB of text — a 10× increase — by adding three additional high-quality corpora:

Corpus	Size	Domain
Wikipedia (English)	~13 GB	Encyclopaedic, formal
BooksCorpus	~6 GB	Fictional and narrative prose
CC-News	~76 GB	News articles, 2016–2019
OpenWebText	~38 GB	Web text curated from Reddit links
Stories (CommonCrawl subset)	~31 GB	Story-like web content

Critically, RoBERTa demonstrates that training on more data consistently improves performance, even with the same architecture and similar training steps. This finding contributed to the broader trend of ever-larger pre-training corpora in subsequent models (GPT-3, LLaMA, etc.).

Larger Batch Sizes

BERT was pre-trained with a batch size of 256 sequences. RoBERTa experiments with batch sizes of 2K and 8K, finding that larger batches improve perplexity and downstream task performance when combined with appropriate learning rate scaling. Training with large batches is also more computationally efficient on distributed hardware — fewer synchronisation steps are needed for the same number of training tokens.

Longer Sequences

The original BERT paper used shorter 128-token sequences for the majority of pre-training and switched to 512-token sequences only for the final 10% of training. RoBERTa uses full-length 512-token sequences throughout training, providing more long-range context during every training step. This is more computationally expensive (self-attention complexity scales quadratically with sequence length) but consistently improves performance on tasks requiring long-range understanding.

RoBERTa vs. BERT Performance Comparison (GLUE Dev Set)

BERT-Large: GLUE score 80.5

RoBERTa: GLUE score 88.5 (+8 absolute points — same architecture, better training)

Key takeaway: 8 GLUE points of improvement from training procedure changes alone.

This demonstrates that training methodology can be as impactful as architecture design.

RoBERTa held state-of-the-art on multiple GLUE tasks for over a year after its release.

5 ALBERT: A Lite BERT for Self-Supervised Learning of Language Representations

ALBERT (Lan, Chen, Goodman, Gimpel, Sharma & Soricut, Google Research & Toyota Technological Institute at Chicago, 2019) tackles a different problem from RoBERTa. While RoBERTa focuses on improving BERT's performance by training harder and longer, ALBERT focuses on parameter efficiency — achieving competitive or superior performance with far fewer parameters, enabling larger effective model depth within the same memory budget.

ALBERT introduces two parameter-reduction techniques — factorised embedding parametrisation and cross-layer parameter sharing — and replaces the weak NSP objective with a stronger self-supervised task called Sentence Order Prediction (SOP). Together, these innovations allow ALBERT-xxlarge (235M parameters) to outperform BERT-Large (340M parameters) on multiple benchmarks, despite having 30% fewer parameters.

5.1 Factorised Embedding Parametrisation

In standard BERT, the vocabulary embedding size E is tied to the hidden layer size H (both are 768 in BERT-Base or 1024 in BERT-Large). This coupling is inefficient for two reasons:

- Token embeddings need to capture context-independent representations — they are essentially a lookup table. They do not need to be as high-dimensional as the final hidden states, which encode rich contextual information.
- The vocabulary embedding matrix has size $V \times H$ (where $V \approx 30,000$ tokens), representing a large fraction of BERT's total parameters. For BERT-Base: $30,000 \times 768 = 23$ million parameters in embeddings alone — approximately 21% of total parameters.

ALBERT factorises the embedding matrix into two smaller matrices. Instead of projecting from vocabulary one-hot vectors (size V) directly to H , ALBERT first projects to a small intermediate dimension E (set to 128 in all ALBERT configurations), then projects from E to H :

Vocabulary $\rightarrow$ E (small embedding): matrix size $V \times E = 30,000 \times 128 = 3.84\text{M}$ parameters

$E \rightarrow H$ (projection): matrix size $E \times H = 128 \times 768 = 98\text{K}$ parameters (BERT-Base size)

Total embedding parameters: $3.84\text{M} + 0.098\text{M} = 3.94\text{M}$ vs. 23.04M in BERT-Base. This is a $5.8\times$ reduction in embedding parameters for BERT-Base size, growing to $6\times$ for BERT-Large and even larger for models with bigger hidden dimensions. Crucially, because E is the same for all ALBERT configurations, the vocabulary embedding cost does not grow when H is increased — enabling wider or deeper models at the same parameter cost.

5.2 Cross-Layer Parameter Sharing

The second and more dramatic parameter reduction technique in ALBERT is cross-layer parameter sharing. In standard BERT, each Transformer layer has its own independent set of parameters — attention weights, feed-forward weights, layer normalisation parameters, and biases. Across L layers, this means L independent copies of these parameter matrices.

ALBERT shares all parameters across all Transformer layers: every layer uses the identical attention and feed-forward weights. This reduces the Transformer layer parameters from $O(L \times H^2)$ to $O(H^2)$, regardless of depth. An ALBERT model with 12 layers and 768 hidden units uses the same number of parameters as a 1-layer BERT-Base model (minus the embedding matrix).

Does Sharing Hurt Performance?

Counterintuitively, cross-layer parameter sharing does not dramatically hurt performance — and in some configurations improves it. ALBERT paper ablations show that sharing only attention parameters (not FFN) or only FFN parameters (not attention) each degrade performance slightly. Sharing all parameters degrades performance further but is still competitive with BERT when the hidden size is increased.

The key insight is that depth (number of processing steps) is more important than having independent parameters at each step. Sharing parameters is analogous to weight-tying in recurrent neural networks — the model can still represent complex functions by iterating the same computation multiple times. The shared layers effectively act as a learned iterative refinement procedure on the input representations.

Sentence Order Prediction (SOP)

ALBERT replaces BERT's NSP objective with Sentence Order Prediction (SOP). Where NSP distinguishes between consecutive and random sentence pairs (a topic-matching task, not a discourse-structure task), SOP distinguishes between two ordering of the same consecutive sentences:

- **Positive examples:** Sentence A followed by Sentence B (natural order from the document).
- **Negative examples:** Sentence B followed by Sentence A (swapped order).

Because both orderings use sentences from the same document, topic matching is not a valid solution — the model must genuinely learn to identify discourse coherence and natural sentence flow. This is a significantly harder objective than NSP, and ALBERT demonstrates that SOP produces better inter-sentence representations, improving performance on tasks like NLI and multi-sentence question answering.

Model	Parameters	Embedding Dim	Hidden Dim	Layers	GLUE Score
BERT-Base	110M	768 (tied)	768	12	79.6
BERT-Large	340M	1024 (tied)	1024	24	80.5
ALBERT-Base	12M	128	768	12	80.1
ALBERT-Large	18M	128	1024	24	82.3
ALBERT-XLarge	60M	128	2048	24	86.0
ALBERT-XXLarge	235M	128	4096	12	90.9

6 DistilBERT and Knowledge Distillation

DistilBERT (Sanh, Debut, Chaumond & Wolf, Hugging Face, 2019) is a compressed version of BERT produced using a technique called knowledge distillation. While ALBERT reduces parameter count through architectural sharing, DistilBERT reduces model size through post-hoc compression — training a smaller 'student' model to replicate the behaviour of a larger pre-trained 'teacher' model. DistilBERT retains 97% of BERT's language understanding capabilities while being 40% smaller and 60% faster.

6.1 Knowledge Distillation: The Core Idea

Knowledge distillation (Hinton, Vinyals & Dean, 2015) is a model compression technique based on the observation that the soft probability outputs of a trained neural network — not just the hard class predictions — encode rich information about the model's learned representations. A large trained model (the teacher) outputs class probabilities for each example: not just 'this is a cat' but 'this is 87% cat, 8% dog, 3% tiger, 2% other'. These soft probabilities capture inter-class similarities and relationships that are not present in hard one-hot labels.

A smaller student model is then trained to minimise a combined loss that includes both the standard task loss (matching hard ground-truth labels) and a distillation loss (matching the teacher's soft probability outputs). The distillation loss is computed using a temperature parameter T : the teacher's logits are divided by T before softmax, making the distribution 'softer' (more spread across classes) and encoding more information about which classes the teacher considers similar.

```
# Knowledge Distillation Loss (PyTorch)
import torch.nn.functional as F

def distillation_loss(student_logits, teacher_logits, labels, T=4.0, alpha=0.5):
    # Soft target loss: student mimics teacher's soft distribution
    soft_loss = F.kl_div(
        F.log_softmax(student_logits / T, dim=-1),
        F.softmax(teacher_logits / T, dim=-1),
        reduction='batchmean'
    ) * (T ** 2)

    # Hard target loss: standard cross-entropy with true labels
    hard_loss = F.cross_entropy(student_logits, labels)

    # Combined loss
    return alpha * soft_loss + (1 - alpha) * hard_loss
```

6.2 DistilBERT Architecture and Training

DistilBERT uses the same Transformer encoder architecture as BERT but with half the number of layers: 6 layers instead of 12, with the same hidden size (768) and attention heads (12). The token-type embedding layer is removed (since DistilBERT is not trained with NSP) and the pooler (used for [CLS] classification) is retained for task fine-tuning.

The student model is initialised from alternating BERT-Base layers (layers 0, 2, 4, 6, 8, 10 of BERT become layers 0–5 of DistilBERT), taking advantage of the fact that BERT's intermediate representations are already useful starting points. The model is then trained using three combined objectives:

- **MLM Loss:** Standard masked language model loss, as in BERT pre-training.
- **Distillation Loss:** KL divergence between student and teacher (BERT-Base) MLM output distributions, using temperature $T=4$.
- **Cosine Embedding Loss:** Cosine similarity between student and teacher hidden state vectors, encouraging the student to learn similar internal representations, not just similar outputs.

6.3 Performance and Deployment Trade-offs

Metric	BERT-Base	DistilBERT	Change
Parameters	110M	66M	−40%
Inference Speed	1×	1.6× faster	+60%
GLUE Score	79.6	77.0	−2.6 pts
SST-2 (Sentiment)	93.5	91.3	−2.2 pts
SQuAD v1.1 F1	88.5	86.9	−1.6 pts
Model Size on Disk	438 MB	263 MB	−40%

DistilBERT's performance profile makes it the practical choice for production NLP systems where latency, memory, or energy consumption are constraints. Mobile applications, browser-based inference (using ONNX or TensorFlow.js), real-time API services with high request volumes, and edge deployment scenarios all benefit significantly from DistilBERT's reduced footprint with minimal accuracy cost.

When to Choose DistilBERT vs. BERT vs. RoBERTa

DistilBERT: latency-critical production systems, mobile/edge deployment, high-throughput APIs, prototyping.

BERT-Base: balanced accuracy/speed; a strong baseline for research; good for most academic tasks.

BERT-Large: maximum accuracy for offline or batch inference; benchmarking; high-stakes decisions.

RoBERTa: when GLUE/NLU performance is the primary goal; typically the best fine-tuning starting point.

ALBERT-XXLarge: when parameter count must be minimised while maintaining maximum capacity.

7 BERT-Based Applications In Depth

The true measure of BERT's impact lies not in benchmark scores but in its deployment across virtually every domain of applied NLP. Within eighteen months of its release, BERT or its variants were powering production systems at Google, Microsoft, Amazon, Baidu, and hundreds of smaller organisations worldwide. This section surveys the major application categories with concrete examples, implementation patterns, and domain-specific variants.

7.1 Search and Information Retrieval

BERT Transforms Google Search (October 2019)

In October 2019, Google deployed BERT to English search queries — the largest change to Google Search in five years.

Example query: 'Can you get medicine for someone pharmacy'

Before BERT: Google ignored 'for someone', returned results about getting your own prescriptions.

After BERT: BERT parsed the prepositional phrase correctly and returned pharmacy proxy pickup information.

BERT now processes roughly 15% of Google's English queries — the complex, conversational queries where understanding prepositions, negations, and sentence structure makes the biggest difference.

BERT is used in information retrieval in two distinct ways. In re-ranking, a standard BM25 or TF-IDF retrieval step identifies a candidate set of documents, and BERT re-ranks them by encoding the (query, document) pair and predicting relevance. In dense retrieval (e.g., DPR — Dense Passage Retrieval, Karpukhin et al., 2020), BERT independently encodes queries and passages into dense vector embeddings and retrieval is performed by maximum inner product search (MIPS) over a pre-computed passage index.

7.2 Biomedical and Clinical NLP

BioMedBERT for Drug Discovery

BioMedBERT (Gu et al., 2021) — also called PubMedBERT — is pre-trained from scratch on 18 billion words

of PubMed abstracts and PubMed Central full-text articles. It outperforms general BERT and BioBERT (which is fine-tuned from BERT, not pre-trained from scratch) on all nine biomedical NLP tasks in the BLURB benchmark, including NER, relation extraction, sentence similarity, and question answering.

AstraZeneca and other pharmaceutical companies use BERT-based models to mine drug-disease relationships

from 30+ million scientific papers — a task infeasible for human experts at scale.

Biomedical NLP presents unique challenges that standard BERT handles poorly: the vocabulary of medical and scientific text differs dramatically from Wikipedia and BooksCorpus. Terms like 'phosphorylation', 'glucocorticoid', 'adenocarcinoma', and gene/protein names (e.g., 'BRCA1', 'TP53') either do not appear in standard BERT's training data or appear rarely. Domain-specific pre-training solves this by building vocabulary and representations from scratch on in-domain text.

Clinical BERT (Huang et al., 2019) follows the same approach but targets clinical notes from electronic health records — a very different distribution from published biomedical literature. Clinical notes are often abbreviated, grammatically informal, and contain institution-specific codes. Clinical BERT achieves strong performance on clinical NER and ICD code prediction tasks where general BERT struggles.

7.3 Legal and Financial NLP

Legal documents present extreme challenges for NLP: very long sequences (contracts, judgements, regulatory filings span thousands of tokens, far exceeding BERT's 512-token limit), highly specialised vocabulary, and logical structure that standard pre-training objectives do not explicitly model. Researchers have developed several BERT variants for the legal domain:

- **Legal-BERT (Chalkidis et al., 2020):** Pre-trained on 12 GB of legal text (EU legislation, contracts, court cases), demonstrating consistent improvements over BERT on legal NER, contract clause classification, and legal judgement prediction.
- **FinBERT (Araci, 2019):** Pre-trained on financial news and earnings call transcripts, achieving strong performance on financial sentiment analysis — an important application for algorithmic trading and risk management.
- **SEC-BERT (Loukas et al., 2022):** Pre-trained on SEC filings (10-K, 10-Q, 8-K forms), with applications in financial information extraction and risk factor analysis.

7.4 Multilingual and Cross-Lingual BERT

mBERT (Multilingual BERT) was released by Google alongside the original English BERT — it is a single BERT model pre-trained on Wikipedia text in 104 languages simultaneously, using a shared WordPiece vocabulary of ~120,000 tokens. Despite never being explicitly trained for cross-lingual transfer, mBERT exhibits remarkable zero-shot cross-lingual generalisation: a model fine-tuned on English NER can be directly applied to German or Chinese NER with competitive performance.

XLm-RoBERTa (Conneau et al., 2020) extends this approach with the RoBERTa training improvements — more data (2.5TB of Common Crawl text in 100 languages), larger batches, dynamic masking, and no NSP. It substantially outperforms mBERT on all cross-lingual tasks and is now the standard starting point for multilingual NLP applications, including low-resource languages where monolingual pre-training data is scarce.

For Indian languages — directly relevant to Gudsky's research context — IndicBERT and MuRIL (Multilingual Representations for Indian Languages, Google, 2021) provide pre-trained models covering Hindi, Bengali, Tamil, Telugu, Kannada, Malayalam, and other scheduled languages. MuRIL was pre-trained on 1 billion sentences from 17 Indian languages and achieves state-of-the-art results on IndicGLUE, the Indian-language NLP benchmark.

7.5 Conversational AI and Question Answering Systems

BERT transformed question answering from a task requiring elaborate pipeline systems (entity detection, document retrieval, candidate span extraction, answer re-ranking) into an end-to-end fine-tuning problem. The UnifiedQA model (Khashabi et al., 2020) fine-tunes T5 (a BERT-style encoder-decoder) on 8 QA datasets in a unified text-to-text format, demonstrating strong zero-shot generalisation to unseen QA formats.

In the Retrieval Augmented Generation (RAG) paradigm — now standard in enterprise QA deployments — a BERT-based Dense Passage Retriever (DPR) indexes an organisation's documents and retrieves relevant passages in response to a query, which are then passed to a generative model (GPT-3, T5, LLaMA,

etc.) for answer synthesis. This architecture combines BERT's retrieval strengths with the fluency of autoregressive generation.

Application Domain	Model Variant	Key Task	Notable Deployment
Web Search	BERT / MRC-BERT	Query understanding, re-ranking	Google Search (2019)
Biomedical NLP	BioMedBERT, BioBERT, Clinical BERT	NER, Relation Extraction, QA	AstraZeneca, NHS Digital
Legal NLP	Legal-BERT	Contract analysis, NER	Luminance, Kira Systems
Financial NLP	FinBERT, SEC-BERT	Sentiment, Risk analysis	Bloomberg, Two Sigma
Multilingual NLP	XLM-RoBERTa, MuRIL	Cross-lingual transfer	Meta AI, Google Translate
Conversational QA	DPR + generative model (RAG)	Open-domain QA	Microsoft Cognitive Search, Azure

8 GLUE and SuperGLUE Benchmarks

Benchmarks play a critical role in NLP research: they provide standardised evaluation protocols that allow direct comparison between models, guide the research community toward important open problems, and serve as empirical evidence for claims of progress. GLUE (General Language Understanding Evaluation) and its successor SuperGLUE are the most influential NLU benchmarks of the transformer era, having driven the design and evaluation of BERT, RoBERTa, ALBERT, XLNet, T5, DeBERTa, and dozens of other models.

8.1 GLUE: General Language Understanding Evaluation

GLUE (Wang et al., 2019) is a collection of nine NLU tasks, aggregated into a single composite score (the simple average of task-specific metrics). It was designed to test a model's general language understanding across diverse linguistic phenomena: grammaticality, sentiment, paraphrase, textual inference, question answering, and coreference. The tasks span multiple domains (news, fiction, social media, Wikipedia) and vary in size from 2,500 to 392,702 training examples.

Task	Abbreviation	Type	Train Size	Metric
Corpus of Linguistic Acceptability	CoLA	Grammaticality	8,551	Matthew's Corr.
Stanford Sentiment Treebank (2 class)	SST-2	Sentiment	67,349	Accuracy
Microsoft Research Paraphrase Corpus	MRPC	Paraphrase	3,668	F1 / Accuracy
Semantic Textual Similarity Benchmark	STS-B	Similarity	5,749	Pearson Corr.
Quora Question Pairs	QQP	Paraphrase	363,849	F1 / Accuracy
Multi-Genre NLI	MNLI	NLI	392,702	Accuracy
Question NLI	QNLI	NLI (QA)	108,436	Accuracy
Recognising Textual Entailment	RTE	NLI	2,490	Accuracy
Winograd NLI	WNLI	Coreference	635	Accuracy

BERT-Large achieved a GLUE score of 80.5 on the leaderboard — a 7.7-point improvement over the best pre-BERT system at the time. RoBERTa subsequently improved this to 88.5. By early 2020, multiple models surpassed the estimated human performance level of 87.1, rendering GLUE largely saturated and motivating the creation of SuperGLUE.

8.2 SuperGLUE: A Harder Benchmark

SuperGLUE (Wang et al., 2020) was designed to be significantly harder than GLUE, focusing on tasks that require more sophisticated reasoning — reading comprehension, natural language inference with multi-hop reasoning, question answering with logical deduction, and coreference resolution in challenging contexts. At the time of SuperGLUE's release, the estimated human performance was 89.8, while the best model achieved approximately 75 — a 15-point gap.

Task	Type	Train Size	Key Challenge
BoolQ	Yes/No QA over passages	9,427	Long passage, indirect answers
CommitmentBank (CB)	3-class textual entailment	250	Presupposition and commitment
Choice of Plausible Alt. (COPA)	Causal reasoning	400	Requires world knowledge
Multi-Sentence Reading Comp. (MultiRC)	Multi-answer RC	5,100	Multiple correct answers per question
Reading Comp. with Commonsense (ReCoRD)	Cloze-style RC	100,730	Commonsense, coreference
Recognising Textual Entailment (RTE)	Binary NLI	2,490	Carried over from GLUE
Winograd Schema Challenge (WSC)	Coreference	554	Requires commonsense for pronoun resolution
Words in Context (WiC)	Word sense disambiguation	5,428	Polysemy across contexts

By mid-2021, SuperGLUE too was approaching saturation — T5-11B and DeBERTa-1.5B both surpassed human performance. This remarkable progression — from BERT achieving 80.5 on GLUE in 2018 to models surpassing humans on both GLUE and SuperGLUE by 2021 — illustrates both the extraordinary pace of progress in NLP and the difficulty of constructing benchmarks that remain challenging in the face of large-scale pre-training.

8.3 Key BERT-Family Results on GLUE and SuperGLUE

Model	Year	GLUE Score	SuperGLUE Score	Parameters
Human Baseline	—	87.1	89.8	—
BERT-Large	2018	80.5	—	340M
XLNet-Large	2019	88.4	—	340M
RoBERTa-Large	2019	88.5	84.6	355M
ALBERT-XXLarge	2019	90.9	90.9	235M
DeBERTa-Large	2020	91.1	90.3	390M
T5-11B	2020	89.7	90.3	11B

It is worth reflecting critically on what benchmark saturation means. Surpassing human performance on GLUE or SuperGLUE does not mean that machines 'understand' language better than humans — it means that machines have become very good at the specific patterns of reasoning tested by these benchmarks. Adversarial NLI (ANLI) and HELM (Holistic Evaluation of Language Models) were created precisely to probe model capabilities beyond what saturated benchmarks can reveal.

9 Research Frontiers: Probing BERT — What Does It Know?

Having established what BERT is trained to do (Section 1–2), how it is applied (Sections 3–7), and how it is evaluated (Section 8), we turn to the most intellectually rich question: what has BERT actually learned? This question drives a substantial and growing sub-field of NLP research known as interpretability and probing analysis. Understanding the internal representations of large pre-trained models is not merely an academic curiosity — it is essential for diagnosing failure modes, improving architectures, ensuring fairness, and building justified trust in model predictions.

Research Spotlight: BERT Knows Syntax (Jawahar et al., 2019)

Probing experiments reveal a hierarchical organisation of BERT's representations:

- Lower layers (1–4): Capture phrase-level syntax — POS tags, chunk boundaries.
- Middle layers (5–8): Capture syntactic tree structure — constituent phrases, dependency relations.
- Upper layers (9–12): Capture semantic information — coreference, semantic role labelling.

This hierarchical organisation was never explicitly trained — it emerged spontaneously from the MLM objective applied to billions of text tokens. The model discovered the hierarchy of linguistic structure as a consequence of learning to predict masked words in context.

9.1 Probing Classifiers: The Methodology

Probing classifiers (also called diagnostic classifiers) are the primary tool for understanding what information is encoded in the hidden representations of a pre-trained model. The procedure is as follows: take the frozen hidden states from a specific layer of BERT for a set of input sentences; train a simple linear classifier (logistic regression or a shallow MLP) on these representations to predict a specific linguistic property (POS tags, parse depth, NER labels, semantic role, etc.); evaluate the classifier's accuracy.

The key design choice — using a simple classifier — is intentional. A complex classifier with many parameters could learn the linguistic property from scratch, telling us nothing about what BERT has encoded. A linear probe can only succeed if the relevant information is already linearly separable in BERT's representation space. High linear probe accuracy therefore implies that BERT has encoded the property in a structured, accessible form.

```
# Probing classifier example: POS tagging from BERT layer 4
from transformers import BertModel, BertTokenizer
from sklearn.linear_model import LogisticRegression
import numpy as np

bert = BertModel.from_pretrained('bert-base-uncased', output_hidden_states=True)
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')

def get_layer_repr(sentence, layer=4):
    inputs = tokenizer(sentence, return_tensors='pt')
    with torch.no_grad():
        outputs = bert(**inputs)
    return outputs.hidden_states[layer].squeeze(0).numpy() # shape: [seq_len, 768]
```

```
# Train probe: X = BERT hidden states, y = POS tags for each token
probe = LogisticRegression(max_iter=1000)
probe.fit(X_train, y_train) # X from get_layer_repr(), y from Penn Treebank
print(f'Layer 4 POS tagging accuracy: {probe.score(X_test, y_test):.3f}')
```

9.2 What BERT Knows: Empirical Findings

Syntactic Knowledge

BERT encodes substantial syntactic knowledge despite never being explicitly trained on syntactic annotations. Syntactic dependencies — which word governs which other word in a sentence — can be recovered from BERT's attention weights with surprisingly high accuracy. Hewitt & Manning (2019) demonstrated that BERT's representations encode a geometry in which syntactic distance (number of edges separating two words in the parse tree) corresponds to Euclidean distance in the representation space — what they call a 'structural probe'.

Semantic Knowledge

BERT's upper layers encode rich semantic information. Coreference — the relationship between 'The professor announced that she would delay the exam' and the fact that 'she' refers to 'the professor' — is recoverable from BERT representations with high accuracy. Semantic role labelling (identifying who did what to whom in a sentence) achieves near state-of-the-art performance with a simple linear probe on BERT's upper layers.

World Knowledge

Petroni et al. (2019) demonstrated that BERT encodes factual world knowledge in a form accessible to simple queries. Using 'fill-in-the-blank' prompts (e.g., 'The capital of France is [MASK].'), BERT correctly answers 31% of factual queries from the T-REx knowledge base — a knowledge-base completion task it was never explicitly trained on. This suggests that language model pre-training incidentally encodes encyclopaedic knowledge as a side effect of modelling language.

9.3 What BERT Does Not Know: Limitations

Despite its impressive representational capabilities, BERT exhibits systematic weaknesses that have motivated subsequent architectural and training innovations:

- **Counting and numerical reasoning:** BERT performs poorly on tasks requiring precise counting or arithmetic, reflecting the limitations of modelling numbers as tokens rather than values.
- **Long-range dependencies beyond 512 tokens:** BERT's fixed 512-token context window makes it poorly suited for long documents without chunking or hierarchical approaches.
- **Compositional generalisation:** BERT struggles with systematic compositionality — combining known concepts in novel configurations. SCAN and COGS benchmarks demonstrate that BERT fails to generalise compositionally in the way humans do.
- **Logical reasoning and negation:** BERT often ignores negation ('not', 'never', 'hardly') in NLI tasks, predicting entailment even when a negated sentence should predict contradiction. This reflects a bias toward lexical overlap rather than genuine logical reasoning.
- **Temporal reasoning:** BERT has limited ability to reason about event timelines and temporal ordering, since its pre-training objective does not explicitly reward temporal understanding.

9.4 Attention Analysis: What Does BERT Attend To?

Visualising BERT's attention weights has become a popular interpretability technique. Clark et al. (2019) systematically analysed attention patterns across all heads in BERT-Base and found that different heads specialise in different linguistic relationships:

- **Direct object head:** Several heads in layer 8–10 attend from verbs to their direct objects.
- **Coreferent mention head:** Certain heads in layer 5–7 attend from pronouns to their antecedents.
- **[CLS] and [SEP] attention:** A large fraction of attention mass in many heads is directed toward [CLS] and [SEP] tokens — possibly a 'no-op' pattern allowing heads to attend to nothing useful while minimising loss.
- **Syntactic dependency heads:** Multiple heads track specific grammatical dependencies — subject-verb agreement, prepositional attachment, noun phrase structure.

However, attention weight analysis must be interpreted cautiously. Jain & Wallace (2019) and Wiegrefe & Pinter (2019) demonstrate that attention weights are not reliable explanations of model predictions — high attention weight on a token does not necessarily mean that token drove the model's output. Attention is one component in a complex multi-layer computation; gradient-based attribution methods (Integrated Gradients, SHAP) provide complementary and often more reliable explanations.

9.5 Implications for Research and Practice

The interpretability literature on BERT has several direct implications for researchers and practitioners building NLP systems:

- **Layer selection for feature extraction:** If using BERT's representations without fine-tuning, extracting from the upper layers (9–12) captures the richest semantic information for most NLU tasks. For syntactic tasks, middle layers (5–8) may be more appropriate.
- **Diagnosing model failures:** When a BERT-based model fails on a specific input type (negations, long sentences, numbers), probing experiments can identify which representational capability is missing or insufficient, guiding targeted improvement.
- **Bias analysis:** Probing classifiers can detect whether BERT's representations encode demographic biases (gender, race, nationality) that might cause discriminatory predictions in downstream tasks. This is an active area of research in responsible AI.
- **Architecture decisions:** Understanding that upper layers are more task-specific and lower layers are more general has informed decisions about which layers to freeze vs. fine-tune and how to construct effective multi-task learning systems.

Research Direction: BERT as a Scientific Tool

An emerging research direction uses BERT as a tool for scientific discovery, not just language processing. BioLinkBERT (Yasunaga et al., 2022) is pre-trained on PubMed with linked citations, learning to connect findings across papers. It achieves state-of-the-art on MedQA and PubMedQA — tasks that require synthesising knowledge across multiple scientific publications. MatBERT (Walker et al., 2021) is pre-trained on materials science literature and can extract synthesis procedures and material properties

directly from research papers — accelerating materials discovery for applications in clean energy, semiconductors, and pharmaceuticals.

9.6 Summary: BERT, RoBERTa, ALBERT, and DistilBERT at a Glance

Model	Year	Key Innovation	Best Use Case	Limitation
BERT	2018	Bidirectional MLM + NSP pre-training	General NLU, fine-tuning baseline	NSP is weak; undertrained
RoBERTa	2019	Dynamic masking, no NSP, 10× more data	Best general-purpose NLU model	Large compute for pre-training
ALBERT	2019	Factorised embeddings, shared layers, SOP	Parameter-efficient large models	Slower inference (shared layers iterate)
DistilBERT	2019	Knowledge distillation, 40% smaller	Production deployment, mobile, edge	Minor accuracy degradation
mBERT	2018	104-language joint pre-training	Cross-lingual zero-shot transfer	Lower per-language performance than monolingual models
XLM-RoBERTa	2020	RoBERTa training on 100 languages, 2.5TB	Multilingual, low-resource languages	Large model size

The future of AI is bright, and you can be part of it!